

OPERIX

Debian com Docker

Instalação, operação, backup e restauração

GRAFmarques • INFinni • M.Print

Gestão operacional multiempresa

Versão 1.01-2026

Data de revisão: 21/08/2026

Objetivo

Executar Nginx, API Express e PostgreSQL em containers na rede local. Apenas a porta web é publicada. O banco permanece em uma rede Docker interna.

Requisitos

- Debian 11, 12 ou 13 compatível com a versão atual do Docker Engine.
- Docker Engine, Buildx e plugin Docker Compose v2.
- Pelo menos 4 GB de RAM, 2 núcleos e 20 GB livres para homologação.
- IP fixo ou reserva DHCP para o servidor.
- Acesso à internet durante a primeira construção das imagens.

Use a instalação oficial do Docker Engine para Debian:

- <https://docs.docker.com/engine/install/debian/>
- <https://docs.docker.com/compose/install/linux/>

O plugin atual utiliza o comando `docker compose`. Não use o executável legado `docker-compose`.

Preparar os arquivos

Extraia `Operix-Debian-Docker-v1.01-2026.zip` em `/opt/operix` e entre na pasta:

```
cd /opt/operix
cp .env.example .env
nano .env
```

Defina:

- `POSTGRES_PASSWORD`: senha exclusiva e forte do banco.
- `JWT_SECRET`: chave aleatória com pelo menos 64 caracteres.
- `WEB_PORT`: porta da rede local, normalmente 8080.
- `APP_ORIGIN`: endereço completo usado pelos navegadores, por exemplo `http://192.168.0.10:8080`.

Para gerar uma chave JWT:

```
openssl rand -hex 48
```

Se a senha do PostgreSQL contiver caracteres especiais reservados de URL, use uma versão codificada ou uma senha alfanumérica longa, pois ela compõe DATABASE_URL.

Primeira inicialização

```
chmod +x scripts/debian/*.sh
./scripts/debian/instalar.sh
```

O script valida Docker, Compose, .env e valores de exemplo antes de executar:

```
docker compose up --build -d
```

Acesse:

```
http://IP-DO-SERVIDOR:8080
```

O banco novo cria Grafmarques, INFINNI e M.Print. O usuário inicial de demonstração da API é `admin@operix.local`, com senha `TroqueAgora!2026`; essa credencial deve ser alterada antes de qualquer uso real.

A versão entregue mantém a avaliação e sua justificativa como obrigatórias antes da abertura de outro chamado pelo mesmo solicitante. Não remova essa validação ao integrar novos clientes do frontend.

Verificação

```
./scripts/debian/verificar.sh
docker compose logs --tail=200 api web db
```

Estados esperados: `operix-db`, `operix-api` e `operix-web` em execução e saudáveis.

Backup

```
./scripts/debian/backup.sh
```

O arquivo compactado será criado em `backups/` com permissão restrita. Copie os backups para outro equipamento e, em produção, proteja-os com criptografia.

Exemplo de agendamento diário às 02:15 com cron:

```
15 2 * * * cd /opt/operix && ./scripts/debian/backup.sh >> /var/log/operix-backup.log 2>&1
```

Restauração

Primeiro faça outro backup. Depois execute:

```
./scripts/debian/restaurar.sh backups/operix-AAAAMDD-HHMMSS.dump
```

O script exige a confirmação textual RESTAURAR porque substitui os dados atuais.

Atualização

Coloque os arquivos novos sobre uma cópia de trabalho e execute:

```
./scripts/debian/atualizar.sh  
./scripts/debian/verificar.sh
```

O script cria backup antes de reconstruir as imagens. Migrações futuras de banco deverão usar controle de versão; não reaplique manualmente todas as migrações em uma base existente.

Rede e firewall

- Libere WEB_PORT somente para a sub-rede local autorizada.
- Não publique a porta 5432 do PostgreSQL.
- O Docker pode interferir em regras do UFW; valide regras na cadeia DOCKER-USER, conforme a documentação oficial.
- Para acesso fora da rede local, use VPN. Não exponha diretamente o serviço à internet.
- Em ambiente definitivo, coloque HTTPS em um proxy reverso com certificado interno.

Encerramento e diagnóstico

```
docker compose stop  
docker compose start  
docker compose restart api  
docker compose ps  
docker compose logs -f --tail=100
```

`docker compose down` remove containers e redes, mas preserva o volume nomeado do banco. Nunca use `docker compose down -v` em uma instalação com dados que devam ser mantidos.